

LABORATORIO DI AMMINISTRAZIONE DI SISTEMA

PROGETTO

KOUAMOU ROUSSEL

897277

**IMPLEMENTAZIONE INFRASTRUTTURA DOCKER
E SISTEMA DI BACKUP AUTOMATIZZATO**

INTRODUZIONE

Il seguente progetto non è altro che un case-study di un sistema di staging configurato per l'hosting e la gestione di container in un ambiente Linux. Il sistema in questione ha come scopo principale quello di fornire agli sviluppatori un ambiente di test isolato, replicabile e affidabile basato su **Docker**. L'adozione di un'architettura a container permette di superare i classici problemi legati alle discrepanze delle versioni delle dipendenze (il noto problema del "works on my machine"), ottimizzando l'utilizzo delle risorse hardware rispetto alla tradizionale virtualizzazione basata su hypervisor.

Oltre all'implementazione dell'infrastruttura di virtualizzazione a livello di sistema operativo, l'environment è corredato da un solido sistema di **Backup** automatizzato. Tale sistema è stato progettato e implementato per garantire la sicurezza e l'integrità dei dati persistenti (nello specifico, i volumi Docker e i dump dei database relazionali). Nel seguente documento verranno analizzati i servizi utilizzati in questo ambito, fornendo una procedura step-by-step utile a riprodurre fedelmente la configurazione descritta.

DEFINIZIONE DEL PROGETTO (SCENARIO)

Lo scenario ipotetico prevede una startup IT in forte fase di espansione che sta migrando la propria architettura monolitica verso un'architettura a microservizi. Per supportare questo cambiamento e integrare i flussi di Continuous Integration e Continuous Deployment (CI/CD), si rende necessaria la configurazione di un server centrale dedicato all'ambiente di *staging*, installato con sistema operativo Ubuntu Server 22.04 LTS.

Su questo server verranno ospitati molteplici servizi tramite container. Per la presente dimostrazione, lo stack applicativo prevede l'esecuzione di un server web proxy (Nginx) e di un database relazionale (PostgreSQL). Trattandosi di un ambiente in cui vengono caricati dati di test prossimi alla produzione, è di fondamentale importanza che i dati generati e memorizzati all'interno dei container non vadano persi a causa di guasti hardware, corruzioni logiche del file system o errori umani. Per questo motivo, viene implementata una policy di backup ricorrente che esporti a caldo i dati su una partizione di storage separata e dedicata, in previsione di una successiva replica off-site (es. verso un NAS aziendale o uno storage in cloud).

INFRASTRUTTURA UTILIZZATA E PARTIZIONAMENTO

Per simulare un ambiente professionale, il server virtuale (o fisico) scelto per questo progetto presenta le seguenti caratteristiche hardware ipotetiche:

- **CPU:** 4 vCore a 2.4 GHz
- **RAM:** 8 GB ECC

- **Storage:** 1 disco SSD NVMe da 500 GB
- **Sistema Operativo:** Ubuntu Server 22.04 LTS

Un corretto partizionamento è fondamentale per garantire le performance e la stabilità del sistema host. In un ambiente containerizzato, un'eccessiva scrittura di file log da parte dei processi applicativi o il download incontrollato di immagini Docker possono saturare rapidamente lo spazio su disco. Per mitigare questo rischio ed evitare il crash del sistema operativo, il disco da 500 GB è stato partizionato a livello logico (LVM) come descritto nella tabella seguente:

MOUNT POINT	DIMENSIONE	FILESYSTEM	SCOPO SPECIFICO
/ (Root)	50 GB	ext4	Sistema Operativo, librerie condivise, binari di base e configurazioni di sistema (es. directory <code>/etc</code> e <code>/usr</code>).
<code>/var/lib/docker</code>	200 GB	xfs	Partizione dedicata esclusivamente al demone Docker. Ospita le immagini scaricate, i layer dei container in esecuzione, i volumi per i dati persistenti e i log. Il filesystem XFS è consigliato per l'uso con lo storage driver <i>overlay2</i> . Separare questa directory garantisce che l'esaurimento dello spazio da parte dei container non renda il server inaccessibile via SSH.
<code>/backup</code>	220 GB	ext4	Partizione sicura riservata all'archiviazione dei backup automatizzati (dump SQL compressi). Separare i backup dalla root e da Docker previene la perdita dei dati nel caso in cui le partizioni operative subiscano una corruzione.
swap	8 GB	swap	Spazio di paginazione, dimensionato proporzionalmente alla memoria RAM installata (8 GB) per prevenire errori di <i>Out of Memory</i> (OOM).

CONFIGURAZIONE INFRASTRUTTURA DOCKER

In questo capitolo vengono illustrati tutti i passaggi sequenziali necessari per configurare l'infrastruttura a container, partendo da un'installazione pulita e minimale del sistema operativo.

1. PREREQUISITI E INSTALLAZIONE DEL MOTORE DOCKER

Come prima operazione, è prassi amministrativa aggiornare gli indici dei pacchetti del gestore APT e rimuovere eventuali installazioni residue o non ufficiali fornite dai repository di default di Ubuntu, al fine di evitare conflitti di versione:

```
sudo apt update && sudo apt upgrade -y
sudo apt-get remove docker docker-engine docker.io containerd runc
```

Successivamente, si procede all'installazione delle dipendenze necessarie per permettere ad `apt` di comunicare con i repository tramite protocollo sicuro HTTPS. Inoltre, si scarica la chiave crittografica GPG ufficiale di Docker e si aggiunge la sorgente software al sistema:

```
# Installazione pacchetti di utilità e certificati
sudo apt install apt-transport-https ca-certificates curl gnupg lsb-release -y

# Download e aggiunta della chiave GPG ufficiale
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

# Configurazione del repository Docker "stable"
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Aggiornando nuovamente l'indice di APT, sarà ora possibile installare l'ultima versione stabile del demone Docker (Docker Engine), l'interfaccia a riga di comando (CLI), il runtime Containerd e il plugin essenziale Docker Compose:

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin -y
```

2. CONFIGURAZIONE POST-INSTALLAZIONE ED ESECUZIONE

Per motivi di sicurezza e di comodità operativa, il demone Docker richiede privilegi di amministratore. Per permettere all'utente non privilegiato corrente di eseguire comandi Docker senza dover anteporre continuamente sudo, è necessario aggiungere tale utente al gruppo di sistema `docker`.

```
# Aggiunta dell'utente al gruppo
sudo usermod -aG docker $USER

# Applicazione immediata delle modifiche al gruppo corrente
newgrp docker
```

Per verificare che l'installazione sia andata a buon fine e che i servizi siano regolarmente attivi in esecuzione automatica, possiamo lanciare i seguenti controlli:

```
sudo systemctl enable docker
sudo systemctl status docker | grep "Active:"
docker --version
docker compose version
```

3. DEPLOYMENT APPLICATIVO TRAMITE DOCKER COMPOSE

Dopo aver predisposto il demone Docker, il passo successivo consiste nel fare il deployment dell'ambiente di staging. Al fine di mantenere l'architettura dichiarativa e versionabile (Infrastructure as Code), viene utilizzato **Docker Compose**.

Verrà creata una directory di progetto dedicata e all'interno verrà definito il manifesto YAML che descrive i container, i mapping delle porte e i volumi necessari alla persistenza dei dati.

```
mkdir -p ~/staging-env/html && cd ~/staging-env  
nano docker-compose.yml
```

Il contenuto del file `docker-compose.yml` è configurato come segue:

```
version: '3.8'

services:
  web:
    image: nginx:1.25-alpine
    container_name: web_proxy
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/share/nginx/html
    restart: unless-stopped
    depends_on:
      - db

  db:
    image: postgres:14-alpine
    container_name: staging_db
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: securepassword
      POSTGRES_DB: app_db
    volumes:
      - pg_data:/var/lib/postgresql/data
    restart: unless-stopped

volumes:
  pg_data:
```

Analisi della configurazione:

- **Immagini Alpine:** Sono state preferite immagini taggate come *alpine* in quanto estremamente leggere e sicure, riducendo drasticamente la superficie di attacco e l'occupazione sul disco.
- **Restart Policy:** La regola `restart: unless-stopped` garantisce che, in caso di riavvio improvviso del server o di crash temporaneo di un processo interno, il demone Docker provveda a riavviare automaticamente i container, assicurando così l'alta affidabilità (High Availability).
- **Volumi Nominati:** L'utilizzo di `pg_data` per PostgreSQL assicura che il database scriva i propri dati sul disco host in maniera ottimizzata e protetta all'interno di `/var/lib/docker/volumes`, non andandoli a disperdere tra i layer temporanei del container.

Procediamo all'avvio dell'intero stack applicativo in modalità "detached" (in background):

```
docker compose up -d
```

Il comando ``docker ps`` ci confermerà il corretto avvio di entrambi i servizi assegnando loro indirizzi IP interni sulla rete bridge generata di default.

CONFIGURAZIONE DEL SISTEMA DI BACKUP E RIPRISTINO

Sebbene i volumi (come `pg_data` configurato precedentemente) forniscano persistenza bloccando la distruzione dei dati in caso di rimozione del container, non proteggono i dati da una corruzione del database, da un comando `DROP TABLE` eseguito per errore, o da guasti hardware del disco. Di conseguenza, si rende imperativo implementare una logica di backup logico tramite dump.

1. CREAZIONE DELLO SCRIPT BASH DI BACKUP

Lo script si occuperà di interfacciarsi con il container PostgreSQL, eseguire il tool `pg_dump` per ottenere il formato testuale dei dati, comprimere l'output per risparmiare spazio sulla partizione dedicata `/backup` e infine cancellare i backup storici che superano la soglia di ritenzione stabilita (es. 7 giorni).

```
sudo mkdir -p /backup/postgres
sudo chown $USER:$USER /backup/postgres
nano ~/scripts/backup_db.sh
```

Codice sorgente dello script `backup_db.sh`:

```
#!/bin/bash

# =====
# Script di Backup PostgreSQL - Dockerized
# =====

BACKUP_DIR="/backup/postgres"
CONTAINER_NAME="staging_db"
DB_USER="admin"
DB_NAME="app_db"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="$BACKUP_DIR/db_backup_$DATE.sql.gz"
RETENTION_DAYS=7

echo "[*] Avvio operazione di backup per il database: $DB_NAME ($DATE)"

# Esecuzione del dump e compressione istantanea tramite pipe logica
if docker exec -t $CONTAINER_NAME pg_dump -U $DB_USER $DB_NAME | gzip >
$BACKUP_FILE; then
    echo "[+] Backup eseguito con successo: $BACKUP_FILE"
else
    echo "[-] ERRORE CRITICO: Fallimento durante il backup del database!" >&2
    exit 1
fi

# Operazione di pruning: pulizia dei backup più vecchi di 7 giorni
echo "[*] Avvio rotazione dei log (Retention: $RETENTION_DAYS giorni)..."
find $BACKUP_DIR -type f -name "*.sql.gz" -mtime +$RETENTION_DAYS -exec rm {} \;

echo "[+] Operazioni concluse con successo."
```

Successivamente si attribuiscono i necessari permessi di esecuzione allo script appena creato:

```
chmod +x ~/scripts/backup_db.sh
```

2. AUTOMAZIONE SCHEDULATA TRAMITE CRONTAB

L'esecuzione manuale dello script non è adatta per ambienti di produzione o staging stabili. Viene quindi utilizzato lo scheduler di sistema `cron` per lanciare il task regolarmente in orari notturni, momento in cui il carico sul database è tipicamente ridotto.

```
crontab -e
```

Aggiungiamo la seguente espressione cronologica:

```
# Esegue lo script ogni giorno alle ore 02:00 di notte
0 2 * * * /home/user/scripts/backup_db.sh >> /var/log/docker_backup.log 2>&1
```

Questa configurazione non solo automatizza il backup, ma reindirizza gli output (sia stdout che stderr) in un file di log persistente, essenziale per il troubleshooting e l'auditing delle operazioni da parte dell'amministratore di sistema.

3. PROCEDURA DI RIPRISTINO (DISASTER RECOVERY)

Un sistema di backup è considerato valido solo se la procedura di ripristino è stata testata con successo. In caso di necessità (ad esempio, un database corrotto nell'ambiente di staging), il ripristino da uno dei file archiviati nella directory `/backup/postgres` si effettua scompattando l'archivio ``gzip`` e iniettando direttamente le istruzioni SQL nel container in esecuzione.

Esempio pratico di comando di ripristino dal terminale del server host:

```
# Decomprime l'archivio al volo e lo invia come standard input al client psql
interno al container
gunzip -c /backup/postgres/db_backup_20231024_020000.sql.gz | docker exec -i
staging_db psql -U admin -d app_db
```

L'utilizzo del parametro `-i` (interactive, senza tty allocato) in `docker exec` è fondamentale per permettere la corretta canalizzazione dei dati (pipe) dall'host Linux verso il container.